

BEN REINER

■ Education

- BS Chemistry & Applied Maths, Boston College, 2013
- PhD Organometallic Chemistry, Ohio State University, 2018
- PostDoc Catalysis Informatics, University of Minnesota, 2019



■ Career @ Dow

- **Senior Research Specialist**, Analytical Chemistry, Dow Core R&D, 2020 - 2024
- **Associate Research Scientist**, Information Research, Dow Core R&D, 2024 – 2025
- **Research Scientist**, MiLOS, Dow Core R&D, 2026 – present
 - ✓ Research: machine assisted synthesis, reaction modeling, material omics, spectroscopy & compressed sensing, predictive biodegradation, patent intelligence
 - ✓ Citizen Data Science and Digital Transformation
 - ✓ Digital Synthesis & Catalysis AI
 - ✓ Biomaterials Product Development



Agentic AI for Chemistry

The right tool for the right job

Ben Reiner, Dow · Presenter

With content contributions from James Wade

Created by humans for humans

Key ideas for meaningful agents

1. Foundation

What is your training data, what assumptions are made, and how well is an underlying chemical space represented

Ground truth & data models

2. Right Choice

When and where are agents critical as opposed to alternative mathematical approaches. Rubrics for evaluating data pipelines.

LLMs, statistics, quality metrics

3. Optimizing

Details and interfaces matter. How we construct better prompts, more robust infrastructure, and self-improving pipelines.

Rationally engineered pipelines

PART 1

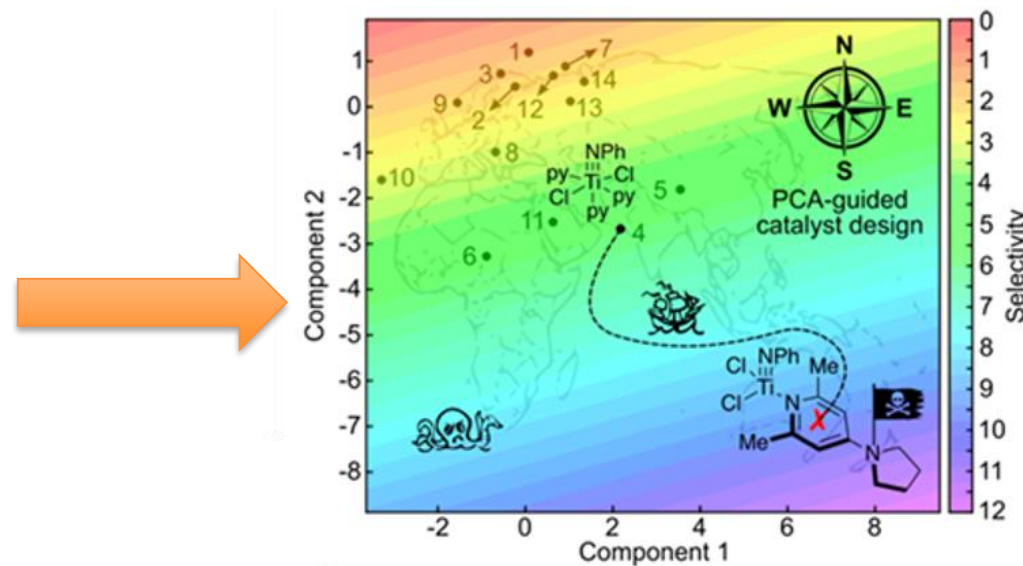
What can't an agent know?

Data and knowledge foundations drive quality models

DEFINITION

Chemical Domains

A formulated abstraction of a physical space a mathematical operator uses for **navigation and context**, while pursuing or optimizing single or multiple objectives.



FOR EXAMPLE

In practice: the available synthetic space for a reaction of choice including conditions, reagents, catalysts, solvents, etc. represented as a matrix or tensor.

Training Set Bias

Ideal data

- **Homogenous**

covers a large, expansive chemical domain, with equal coverage across the space

- **Dense**

each part of the space is heavily covered by real experimental data points

- **Interpolation**

local models capture appropriate nuance and activity cliffs within your chemical space

- **Extrapolation**

new data is at least partially predicted by the existing corpus and can be easily ingested for augmentation

Real data

- **Heterogenous**

covers a small or spotty amount of chemistry with heavy density in only a handful of spaces

- **Sparse**

chemical space is undersampling and critical areas have little or no data

- **Model instability**

poor data quality ensures surrogate models do not capture local structures or clustering

- **New Data Breaking**

new data can't be easily included in the old training or represents a whole new mechanism

Data Visualization

Fluency in a few dimension reduction techniques will dramatically accelerate your ability to predict when an agent will behave...or not

PCA

*principal component
analysis*

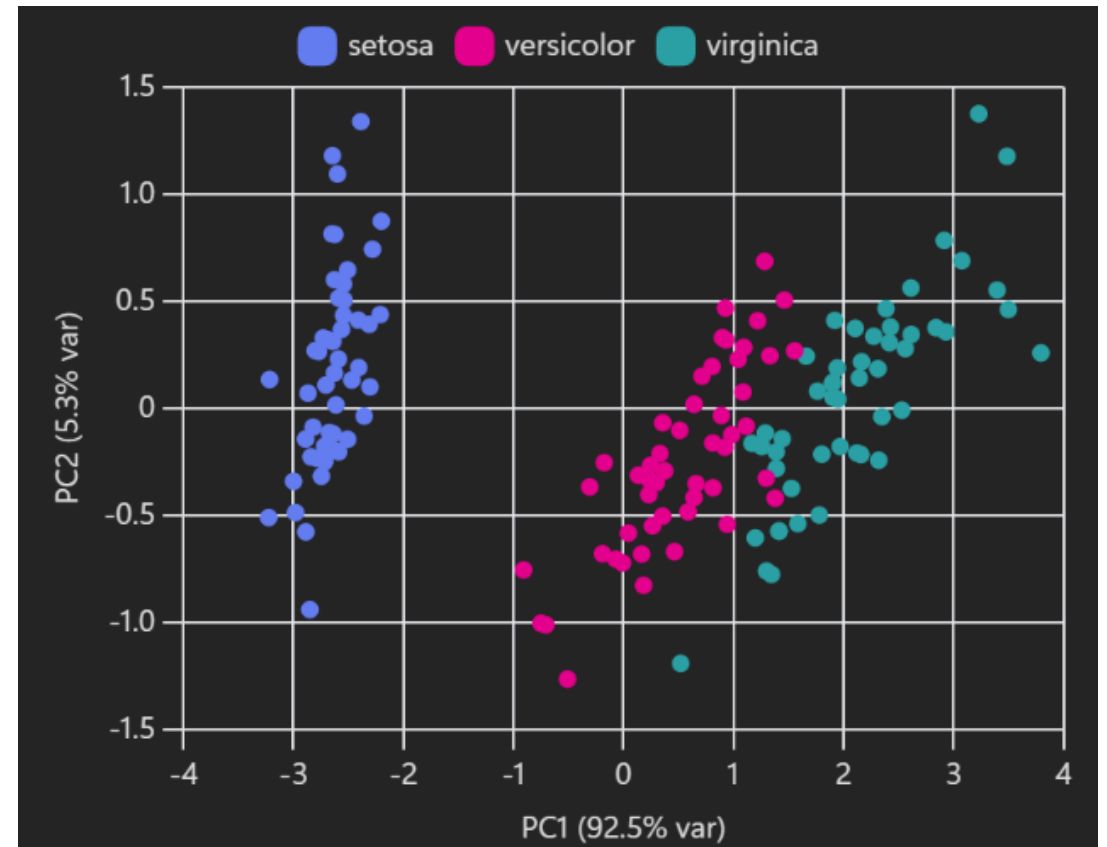
UMAP

*uniform manifold
approximation & projection*

SOM

self-organizing maps

Graphs



Data Visualization

Fluency in a few dimension reduction techniques will dramatically accelerate your ability to predict when an agent will behave...or not

PCA

*principal component
analysis*

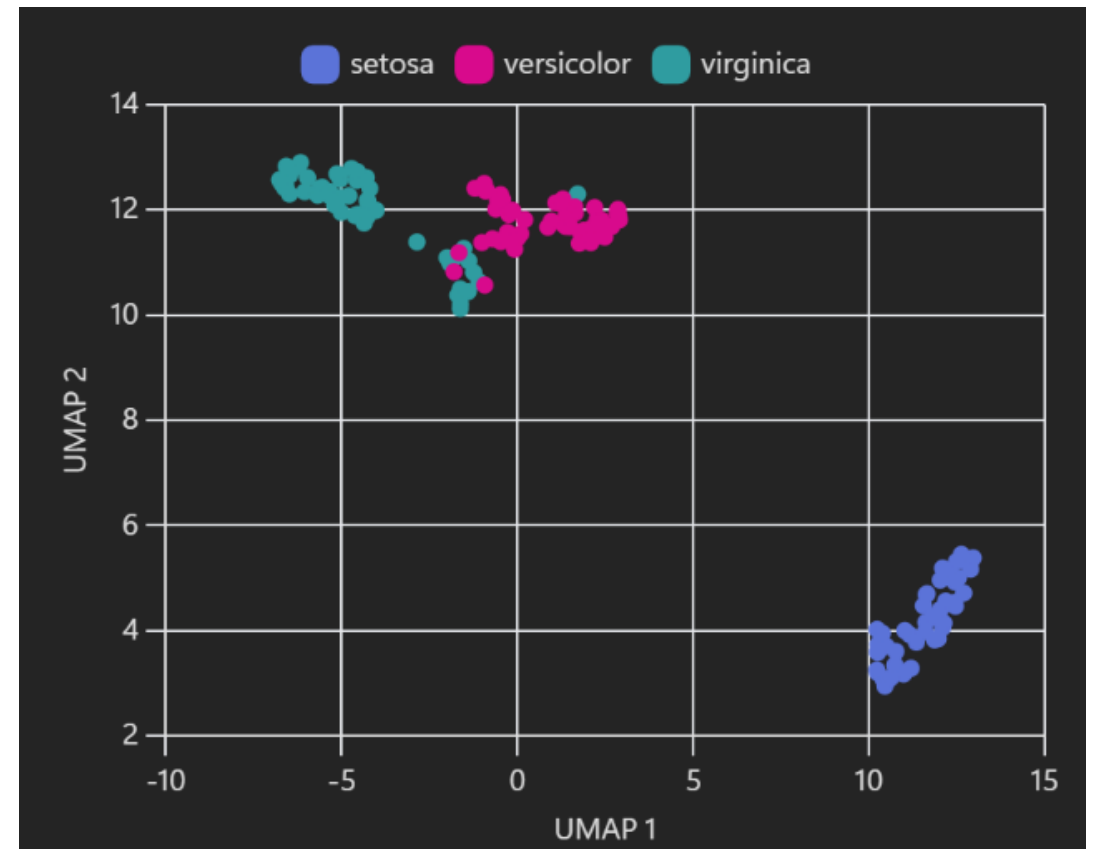
UMAP

*uniform manifold
approximation & projection*

SOM

self-organizing maps

Graphs



Data Visualization

Fluency in a few dimension reduction techniques will dramatically accelerate your ability to predict when an agent will behave...or not

PCA

*principal component
analysis*

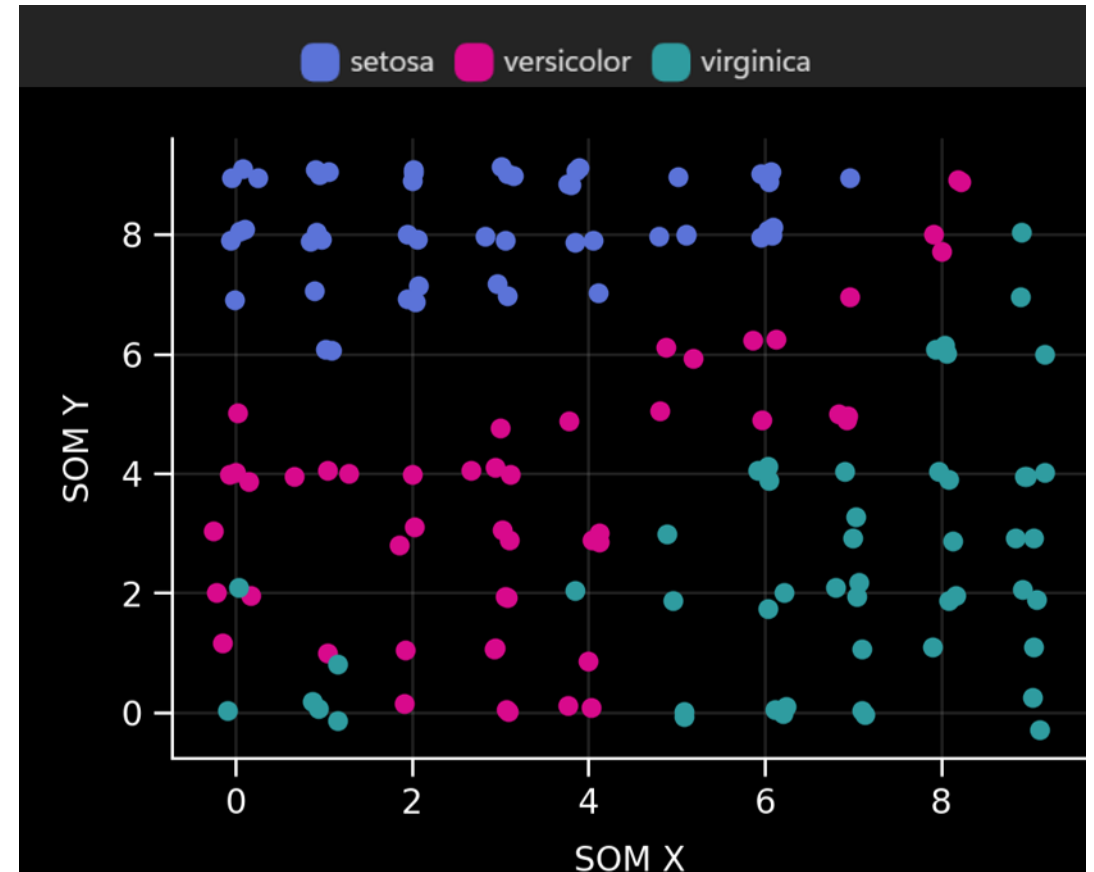
UMAP

*uniform manifold
approximation & projection*

SOM

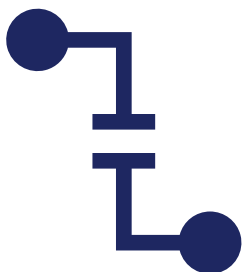
self-organizing maps

Graphs



What's in a Name? Analytical characterization data

Your navigable chemical space is defined by your domain but your objective is defined by your characterization data. **Unknown characterization issues will contaminate any mathematical treatment** of your data.



- ☐ Bad calibration (SEC)
- ☐ **Unknown response factors (LC-MS)**
- ☐ Matrix effects (NMR, FTIR)
- ☐ Data processing variance
- ☐ **Heteroskadicity**



TAKE A MOMENT

Think about all the steps you need to reproduce your chemistry – can you translate that programmatically today?

Self-driving labs (SDLs) require autonomous agents to make decisions without humans

Reaction Robotics

The physical machinery responsible for actually conducting the reactions including weighing reagents and solvents, heating or cooling, mixing, and purifying the crude material.

do the chemistry

Characterization

Sensors and instruments designed to measure expected and unexpected products and impurities. Ideal SDLs will have robots which can sample prep, load, and even optimize the characterization.

see what you made

Decision-Maker

A mathematical operator, with or without human intervention, which can decide which next experiment(s) to pursue based on known objectives or criteria.

plan next best experiment

SDLs Safety Checks & Guardrails

✓ Good practices

+ Physical stops

reactions have overheat protection, robotic arms can not physically swing out of a known space, etc.

+ Human-in-the-loop

human check-in built at key decision points, parallel “race” against human operators to assay bias, etc.

+ Auditable data & decision-making

git and annotation for the code, management of change (MOC) documentation, etc.

✗ Bad practices

— Programmatic stops

csv files containing “bad” reagents, hard coded boiling points, instrument overrides, etc.

— Zero human check or intervention

long running SDLs, no protocols for refilling reagents, no manual audits of data or decisions

— Irreproducible pipelines

worst extraction task; each one becomes an archaeology project

Knowledge check: Data Corruption

Where will an autonomous agent fail when analyzing the data represented in the table below?

reagent	CASRN	amount: mmol	phase	concentration: %	role
toluene	108-88-3	20	liquid	100	solvent
Pd(PPh3)4	108-88-3	0.05	solid	100	catalyst
p-chlorotoluene	106-43-4	0.6	liquid	50	reagent
octylamine	111-86-4	0.5	liquid	40	reagent

PART 2

A Calvacade of Choices

Guidelines for choosing a mathematical approach to chemistry problems

DEFINITION

What is an agent?

An agent is a large language model (LLM), in a harness (programmatic wrapping), that **calls tools repeatedly in a loop**, deciding each next step from the last result.



FOR EXAMPLE

User: Delete all the csv files in the current directory.

Claude: Run the `ls()` tool

ellmer: `a.csv, b.csv, a.R, b.R, c.R`

Claude: `[Run rm("a.csv"), Run rm("b.csv")]`

DEFINITION

ReAct Agent



An agent using reasoning (“re”) and acting (“act”) framework to combine chain of thought reasoning with external tool use to complete complex tasks.

(2) AlfWorld

You are in the middle of a room. Looking quickly around you, you see a cabinet 6, a cabinet 1, a coffee machine 1, a countertop 3, a stove burner 1, and a toaster 1.
Your task is to: Put some pepper shaker on a drawer.

(2a) Act-Only

Act 1: Go to drawer 1
Obs 1: The drawer 1 is closed.

Act 2: Open drawer 1
Obs 2: You open the drawer 1. The drawer 1 is open. In it, you see a dish sponge 2, and a spoon 1.

Act 3: Go to sinkbasin 1
Obs 3: On the sinkbasin 1, you see a dish sponge 3, a spatula 1, and a spoon 2.

Act 4: Take peppershaker 1 from sinkbasin 1
Obs 4: Nothing happens.

Act 5: Take peppershaker 1 from sinkbasin 1
Obs 5: Nothing happens.



(2b) ReAct (Reason + Act)

Act 1: **Think**[First I need to find a pepper shaker...more likely to appear in cabinets {1-6}, countertops {1-3}, ...]
Act 2: Go to cabinet 1
Obs 2: On the cabinet 1, you see a vase 2.

(Here the agent go to cabinet 1, then cabinet 2, 3, then countertop 1 and 2)

Act 6: Go to countertop 3
Obs 6: On the countertop 3, you see a apple 1, a bread 1, a pepper shaker 1, and a vase 3.

Act 7: Take pepper shaker 1 from countertop 3
Obs 7: You pick up the pepper shaker 1 from the countertop 3.

Act 8: **Think**[Now I find a pepper shaker 1. Next, I need to put it in/on drawer 1.]
Act 9: Go to drawer 1
Obs 9: Drawer 1 is closed.

Act 10: Open drawer 1
Obs 10: You open Drawer 1 ...

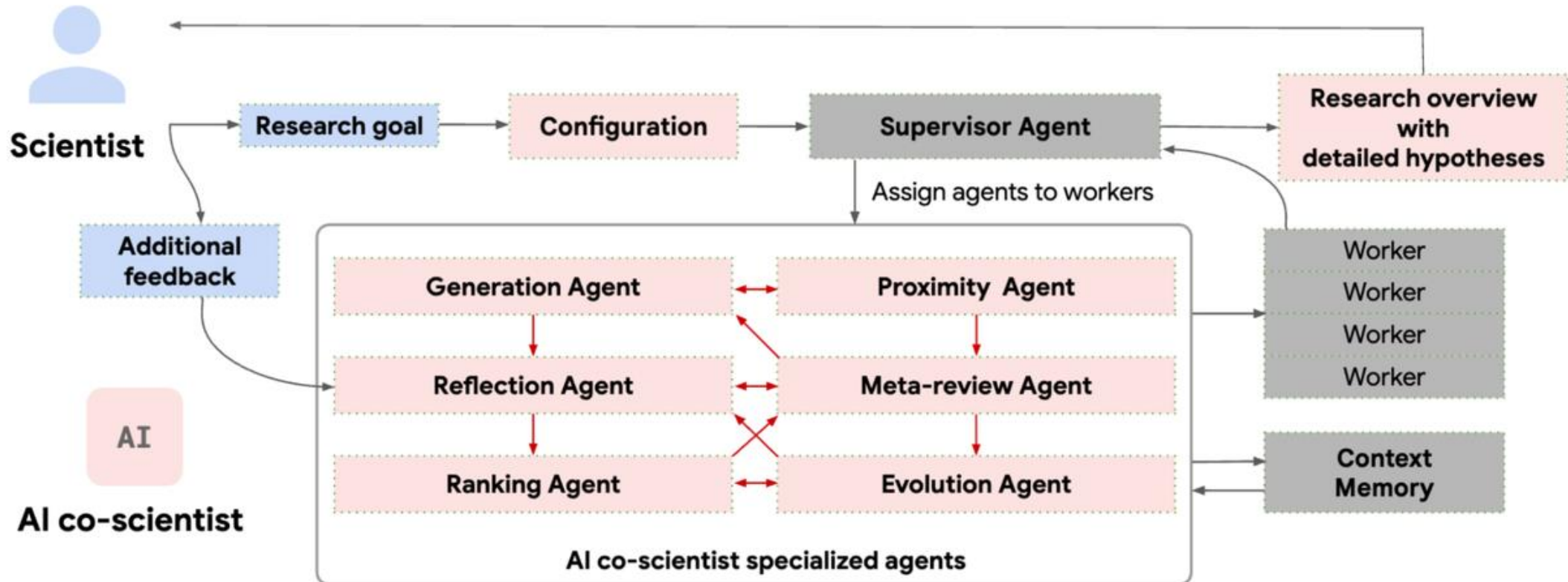
Act 11: Put pepper shaker 1 in/on drawer 1
Obs 11: You put pepper shaker 1 in/on the drawer 1.



DEFINITION

Co-scientist

Multi-agent system built on Gemini 2.0 as an AI collaborator designed to help scientists generate new hypotheses and accelerate discovery



Taxonomy of Problems

Scientific problems exist on a spectrum ranging from inappropriate to perfectly poised for LLM-based agents

A/B Hypothesis Testing

do two catalyst have the statistically different turn over number? does one polymer have disproportionately higher shear strength?

Regression

find the best polynomial fit between heterocycle pKa and reaction rate using a linear least squares regression

Classification

classify reagents into good/medium/bad partners for Negishi couplings

Semantic Analysis

find all mentions of PFAS and their synonyms in the patent and open literature and analyze their occurrence per market segment

TAKE A MOMENT

What is a problem that does not require an LLM “brain”? What is a problem that critically requires one?

Rubric for Choosing the Right Tool

Thinking critically around the “job to be done” will help you build an ideal pipeline

☐

Can I represent the problem mathematically?

matrices are easy to subject to canonical data science pipelines

☐

Does a statistical test already exist?

problems that can be reframed into friendly statistical tests should not require an LLM

☐

Is there an “off the shelf” trained ML model available for the task?

don’t rebuild the wheel especially for computationally expensive efforts

☐

What have other researchers used in the past?

wisdom of the crowd is imperfect but a good canary for problem classification

☐

Who is your solution audience?

does the answer need to make sense to a human, a digital agent, matrix algebra, or all 3?

Knowledge check: Choose Wisely

What is a representative data science tool you would use to approach the 3 problems below?

Problem #1

Optimize the reaction conditions of a Buchwald-Hartwig couplings using a set of 40 phosphine ligands

optimization

Problem #2

Determine the economic impact of patents across a corpus in a particular technology segment

analysis

Problem #3

What are the key factors driving selectivity in a Chan-Lam coupling with reagents above and below 10K g/mol

learning

PART 3

Optimizing prompts & pipelines

How to build better, cheaper, modular tools a stranger can reuse and augment

Robust Pipelines: Industrial Perspective

✓ Good pipelines

+ Env/dependency files

Virtual environments, dependency files, frozen models, served endpoints

+ Clear task delineation

readme files and annotated code which details the purpose and output of each code chunk

+ Efficient and low-cost

models are right-sized to purpose, leverage matched language and tool calls, optimized search algorithms

✗ Bad pipelines

— “Rotting” code

workhorse packages and models regularly update and should not be pipeline breaking

— Hyper-specific tooling/tasks

limited tidy data flows or imperfect abstraction leads to a one tool – one solution problem

— Irreproducible or stochastic

require a data owner for start-up and/or gives different answers every time

Thinking About Cost Models

Sometimes the most expensive part of doing science...is you

R&D

Not all problems require solutions and sometimes the cheapest answer is the one you already have.

does a problem need solving?

AI Agents

Misused, misunderstood, or inappropriate AI pipelines can be worse than having no solution in hand at all.

do people need to solve this?

People

Learning how to integrate human and machine intelligence will ultimately lead to faster discovery.

do I have the right people?

Ideas for Cheaper Agents & Development

Cheaper can be defined by time, \$\$\$, or people labor/stress



Copilot (via Google)

exceedingly fast and cheap for quick code inserts or syntax checks



CLI proxies

add-ons like “rtk” reduce token consumption by compressing commands before reaching the LLM



Model matching

tasks that do not require high reasoning can use low level models such as Claude haiku



Claude skills & Copilot agents

building purpose-built skills can help give long prompts over complex tasks a second life



Designing code for re-use

the cheapest agents can interact with a broader ecosystem built from inception for modularity

DSPy: programs, not prompts

How to build language-model programs you can measure, optimize, and hand to someone else

SIGNATURES

OPTIMIZERS

dspy.Flex

Hand-tuned prompts are not reproducible, and nothing warns you when one stops working

Model drift

Phrasing tuned for one model can stop working the week the provider ships an update. Nothing tells you it degraded.

No transfer

What you work out by hand about one model's quirks does not carry over to the next model or the next task.

No measure

Without a metric and a held-out split, "this prompt is better" is an impression formed on three examples.

DSPy answer is to compile the prompt from a metric and a dataset instead of editing it by hand.

DSPy splits an LM program into four parts you can version, test and optimize

SIGNATURE

What goes in and what comes out, as a typed contract.



MODULE

How the work is done:
Predict, ReAct, RLM, Flex.



METRIC

What counts as a good answer, defined in code.



OPTIMIZER

Searches for a better program under that metric.

You can change one part without rewriting the others. For example:

- swap the module (how prediction is done) and keep the signature (inputs and outputs)
- swap in a cheaper model and rerun the optimizer
- tweak the metric and the entire program updates against it.

That is why a DSPy program survives a new model release without a rewrite.

A signature is a typed contract for the task rather than a prompt for one model

InputField marks what the program is given.

Every **OutputField** is a result that each competing implementation must return.

The **docstring** is the task description, and an optimizer is allowed to rewrite it.

```
class ExtractBoilingPoint(dspy.Signature):  
    """Extract the compound and normal boiling  
    point from the record. Convert to kelvin.  
    Return needs_review=True if the value is  
    missing, conflicting, or only a range."""  
  
    text: str = dspy.InputField()  
    compound_name: str = dspy.OutputField()  
    boiling_point_K: float | None = dspy.OutputField()  
    evidence: str = dspy.OutputField()  
    needs_review: bool = dspy.OutputField()
```

Written feedback beat reinforcement learning on six tasks, using far fewer runs

35×

fewer rollouts than GRPO reinforcement learning, across six tasks

+10%

over MIPROv2, the leading prompt optimizer, and +12% on AIME-2025

~75×

cheaper: Shopify moved a GPT-5 task onto a small Qwen model and optimized it

GEPA reads the trajectories of its own failures and writes the next candidate. That beat thousands of policy-gradient rollouts on the same six tasks.

Run, score with feedback, reflect, propose, keep the winners: that is the whole loop



Two models with different jobs. The student model runs the program you are optimizing. A separate reflection model reads what went wrong and writes the next candidate; it is called only a handful of times, so it costs little. When you are optimizing a small model, reflect with a large one.

The metric is where your chemistry judgement enters, and it returns a reason as well as a score

score decides which candidate wins.

feedback is read by the reflection model. It is how you tell the optimizer what a chemist would object to.

A small per-call penalty makes the optimizer prefer the cheaper program only when accuracy holds.

```
def chemistry_metric(example, pred, ..., program_trace=None):  
    s = component_scores(example, pred)  
    calls = len(program_trace or [])  
    penalty = min(0.10, 0.02 * calls)  
  
    return dspy.Prediction(  
        score=max(0.0, s["scientific"] - penalty),  
        feedback="Numerical parsing and unit "  
                "conversion can be deterministic "  
                "when the text is explicit.",  
    )
```

Weight the parts a reviewer would defend: the value, the compound, the exact supporting substring, and the decision to abstain.

dspy.Flex makes the program's source code the thing being optimized

It starts as a baseline

A Flex is built from a signature and starts as a single `dspy.Predict`, or a `dspy.RLM` if you hand it tools.

GEPA rewrites the source

Instead of only rewriting instructions, GEPA proposes a whole new module: split predictors, plain Python, helper functions.

It runs in a sandbox

Optimizer-written code never runs in your process. It executes in a Deno sandbox; only real model calls bridge back out.

What gets searched is the program itself, and your metric decides which version survives.

A worked example: does boiling-point extraction actually need a language model?

40 training, 15 validation and 20 held-out records of chemical-property statements: explicit kelvin, Celsius that needs converting, distractor numbers, ranges, conflicts and missing values.

All three implementations get the same signature and the same metric. The dataset decides which one wins.

MANUAL PARSER

Regular expressions and arithmetic. Zero model calls, and brittle by construction.

DIRECT PREDICTOR

dspy.Predict sends the signature and the record to the student model. One call each.

FLEX + GEPA

Starts as a predictor; the optimizer may replace the whole module. Zero or more calls.

The metric charges 0.02 per predictor call, capped at 0.10, so a correct one-call answer still beats a wrong zero-call answer.

When the context is too big to read, dspy.RLM has the model explore it with code

PEEK

The model sees metadata and a preview, not the whole text.



EXPLORE

It writes Python in a sandbox to search and filter.



ASK

llm_query() runs a sub-model on the snippets it found.



SUBMIT

It returns typed fields your program can use.

Why this matters in a lab: supporting information runs to hundreds of pages, instrument logs to millions of lines, and a screening table will not fit in a context window. RLM keeps the data in a REPL and loads only the part the model asks for, and you can read the code it ran.

What this buys you: reproducibility, a cheaper model, and an argument you can defend

It is version-controlled

The prompt stops being a paragraph in a group chat. Program, metric and splits live in git and rerun on demand.

Small models get good

An optimized small model often matches a hand-prompted frontier model, which matters when you pay per token out of a grant.

You can defend it

A metric, a held-out split and a printed program are things a supervisor or a reviewer can actually argue with.

When the next model lands, you rerun the optimizer against it instead of rewriting your prompts by hand.

Case studies reported in the DSPy documentation: Shopify (**~75× cheaper, ~2× more reliable**)

References and further reading

- **Google Research**

[Accelerating scientific breakthroughs with an AI co-scientist](#)

- **Wickam, H**

[What is an agent? - by Hadley Wickham](#)

- **Yao, S.**

[REACT: Synergizing Reasoning and Acting in Language Models](#)

- **rtk**

[GitHub - rtk-ai/rtk: CLI proxy that reduces LLM token consumption by 60-90% on common dev commands. Single Rust binary, zero dependencies · GitHub](#)

- **DSPy**

[Agrawal et al., GEPA, arXiv:2507.19457 \(ICLR 2026\) ; DSPy](#)